# MSTest: Examples

Gabriel Rodriguez Rondon
Jean-Marie Dufour
McGill University

29-May-2020

## Installing and Loading MSTest R package

### Install Latest Version of MSTest R Package

The follwoing block of code can be used to download and install the latest version of the MSTest R package from its github repository (https://github.com/roga11/MSTest). Installing packages directly from github requires using *devtools*. The code below will check if devtools is already installed and install it if it is not.

The package does not need to be re-installed every time, just when new updates are avaiable. If the latest version is already installed, proceed to the next section.

```
rm(list=ls())
if (is.element('devtools',installed.packages()[,1])==FALSE){
  install.packages('devtools')
}
devtools::install_github('roga11/MSTest')
```

```
## Downloading GitHub repo roga11/MSTest@master
```

```
##
##      checking for file '/private/var/folders/8q/j2gsms252hvb_8vm50t7qkch0000gn/T/Rtmp1ki1vQ/remotes38
##   -  preparing 'MSTest':
##      checking DESCRIPTION meta-information ...  v  checking DESCRIPTION meta-information
##   -  cleaning src
##      Warning: /private/var/folders/8q/j2gsms252hvb_8vm50t7qkch0000gn/T/RtmphBF5XJ/Rbuild388d3bd6a4ae/
##    Warning: /private/var/folders/8q/j2gsms252hvb_8vm50t7qkch0000gn/T/RtmphBF5XJ/Rbuild388d3bd6a4ae/MS
##   -  installing the package to process help pages
##   -  saving partial Rd database (26.2s)
##   -  cleaning src
##   -  checking for LF line-endings in source and make files and shell scripts
##   -  checking for empty or unneeded directories
##   -  building 'MSTest_1.0.tar.gz'
##
##
```

### Load Package

Once package has been installed it can be loaded.

```
library(MSTest)
```

# Data

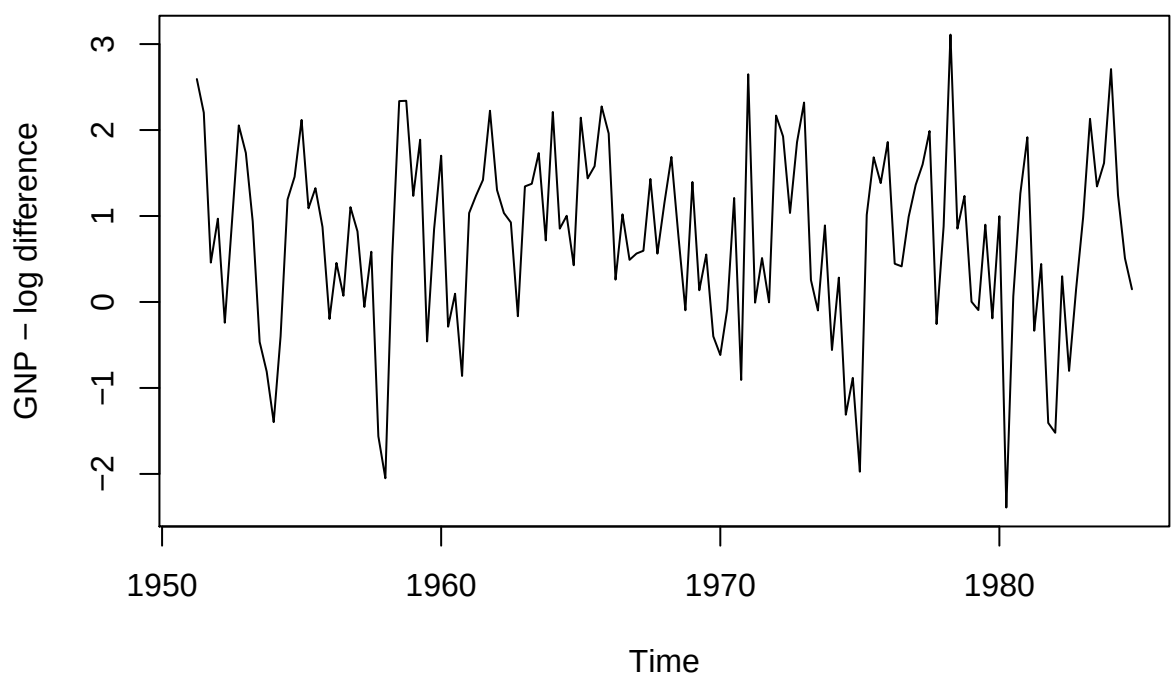
## Load Data

The package includes 3 datasets that can be used as examples. The three datasets are:

- hamilton84GNP: Data from 1951Q2 - 1984Q4 used in Hamilton (1989) and Hansen (1992; 1996)
- carhuplo10GNP: Data from 1951Q2 - 2010Q4 used by Carrasco, Hu and Ploberger (2014)
- kasshi14GDP: Data from 1960Q1 - 2014Q4 used in Kasahara and Shimotsu (2019)

They can be loaded using the following code.

```
GNPdata <- hamilton84GNP # this can be hamilton82GNP, carhuplo10GNP or kasshi14GDP
Y       <- GNPdata$GNP_logdiff
date    <- GNPdata$DATE
plot(as.Date(date),Y,xlab='Time',ylab='GNP - log difference',type = 'l')
```




You an also learn more about these dataset and their sources by using the following line(s) and reading its description in the help tab.

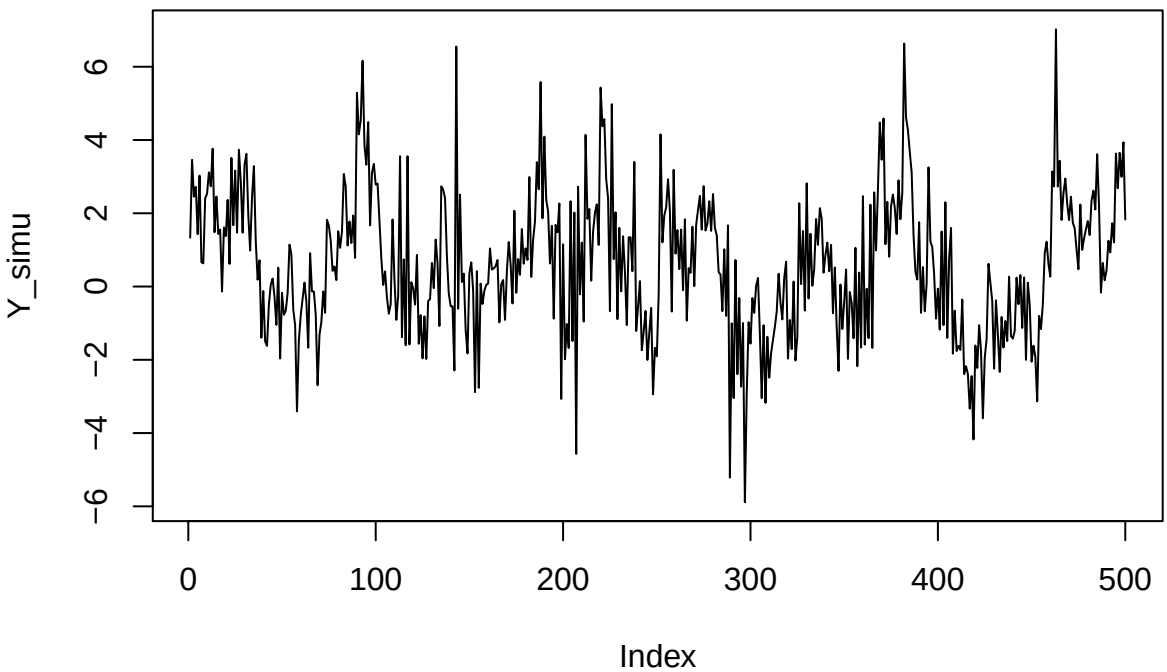
```
?hamilton84GNP
```

## Simulated Data

The function *simuARMS* can be used to simulate an Auto-Regressive Markov-Switching process. It requires the user to specify the sample size $T$, the mean in each regime $\mu_1, \mu_2$, the standard deviation in each regime

$\sigma_1, \sigma_2$, the limiting probabilities of being in each regime $p$ and $q$ and the auto-regressive coefficients $\phi_i$. The parameters should be put together in a single vector in the follwoing order: $\theta = [\mu_i, p, q, \sigma_i, \phi_i]$. The function itself takes the sample size $T$ and the vector $\theta$ as inputs. The following code shows and example of simulating an AR MS process and using the function $ARMSmdl$ to estimating a MS model using this process.

```
n = 500
mu = c(0,2)
std = c(1,2.5)
p = 0.9
q = 0.5
phi = c(0.5,0.25)
th = c(mu,p,q,std,phi)
Y_simu = simuARMS(n,th)
plot(Y_simu, type = 'l')
```




```
arms_mdl = ARMSmdl(Y_simu,ar = 2,k=2)
print("coefficients:")
```

```
## [1] "coefficients:"
```

```
arms_mdl$coef
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.0183474 0.2313576 0.5359689
## [2,] 0.6694047 0.2313576 0.5359689
```

```
print("mu:")
```

```
## [1] "mu:"
```

```r
arms_mdl$coef[,1]/(1-rowSums(arms_mdl$coef[,2:ncol(arms_mdl$coef)]))
```

```
## [1] 0.07885466 2.87701228
```

```r
print("standard dev.:")
```

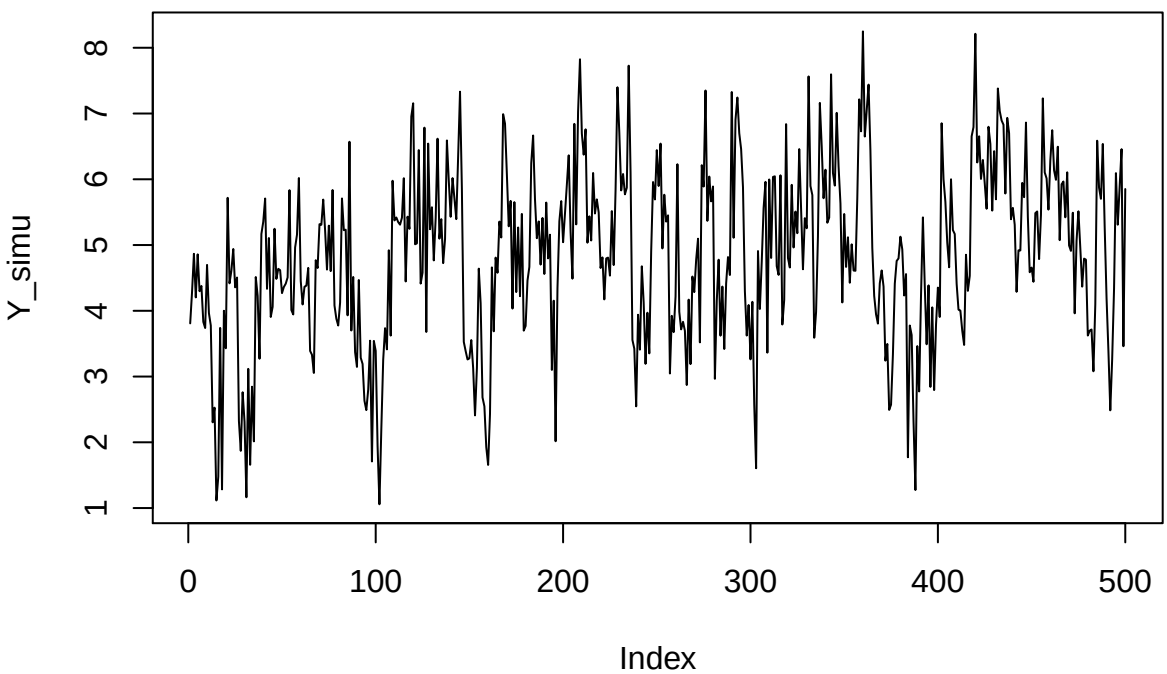
```
## [1] "standard dev.:"
```

```r
arms_mdl$stdev
```

```
## [1] 1.055298 2.324490
```

The function *simuAR* can be used to simulate an AR process. the inputs are sample size $T$, mean $\mu$, standard deviation $\sigma$ and vector of autogressive coefficients $\phi_i$. The following code shows and example of simulating an AR process and using the function *ARmdl* to estimating an AR model using this process.

```r
Y_simu = simuAR(n, mu = 5, std = 1, phi = c(0.5,0.25))
plot(Y_simu, type = 'l')
```




```r
ar_mdl = ARmdl(Y_simu,ar=2)

print("coefficients:")
```

```
## [1] "coefficients:"
```

```r
ar_mdl$coef
```

```
##           [,1]
## [1,] 1.2651810
## [2,] 0.4649435
## [3,] 0.2737895
```

```r
print("mu:")
```

```
## [1] "mu:"
```

```r
ar_mdl$coef[1]/(1-sum(ar_mdl$coef[2:length(ar_mdl$coef)]))
```

```
## [1] 4.842482
```

```r
print("standard dev.:")
```

```
## [1] "standard dev.:"
```

```r
ar_mdl$stdev
```

```
##           [,1]
## [1,] 0.9804888
```

# Hypothesis Testing

## Moment-Based Test

This test is developed by Dufour & Luger (2017)

Optional Arguments are:

- p: number of AR(p) lags (default: 1)
- N: Total number of Monte Carlo simulations for p-val calculations (default: 100 which includes the test-stat)
- N2: Number of simulations for distribution approximation (eq. 16 in Dufour & Luger 2017) (default: 10000)
- N3: Number of values to evaluate for nuissance parameters when using 'randSearch_paramCI' in MaxMC (default: 100000).

### Local Monte-Carlo moment-based test.

This takes a few seconds only.

```r
start <- Sys.time()
DLMCtest(Y, p = 4)
```

```
## $`Test-Stat_min`
## [1] 0.8283654
##
## $`Crit-Values_min`
##     0.90%     0.95%     0.99%
## 0.9877799 0.9933622 0.9996496
##
## $`p-value_min`
## [1] 0.5
##
## $params_min
##         ar1         ar2         ar3         ar4
##  0.30974509  0.12725744 -0.12125820 -0.08922691
##
## $`Test-Stat_prod`
## [1] 0.9612929
##
## $`Crit-Values_prod`
##     0.90%     0.95%     0.99%
## 0.9989143 0.9996247 0.9999983
##
## $`p-value_prod`
## [1] 0.65
##
## $params_prod
##         ar1         ar2         ar3         ar4
##  0.30974509  0.12725744 -0.12125820 -0.08922691
```

```r
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 2.120128 secs
```

Use the following command to learn more from the help tab.

```r
?DLMCtest
```

**Maximized Monte-Carlo moment-based test.**

In addition to parameters above, available optimization methods can be set using 'searchType'. Allowed options are:

- randSearch_paramCI: This searches over the 95% C.I.of consistent esitmate. This gives similar reuslts to MMC results in Dufour & Luger (2017)
- GenSA: Uses Generalized Simulated Annealing optimization algorithm.
- GA: Uses Genetic Algorithms.
- PSO: Uses Particle Swarm Optimization

using $searchType =' randSearch_paramCI'$ produces results coparable to Table 5 of Dufour & Luger (2017) if hamilton82GNP or carhuplo10GNP datasets are used. For this reason, $searchType =' randSearch_paramCI'$ is the default. When using $GenSa$, $GA$ or $PSO$, optimization is performed over entire ar coefficient parameter space (i.e. [-1,1]), but it is constrained to only coefficients for a stationary process.

```r
start <- Sys.time()
DLMMCtest(Y, p = 4, searchType = 'randSearch_paramCI')
```

```
## $`Test-Stat_min`
## [1] 0.5759536
##
## $`Crit-Values_min`
##          0.90%     0.95%    0.99%
## [1,] 0.9785999 0.9875501 0.999104
##
## $`p-value_min`
## [1] 1
##
## $params_min
##            ar1       ar2         ar3        ar4
## [1,] 0.3690076 0.2267961 -0.04045714 -0.1942158
##
## $`Test-Stat_prod`
## [1] 0.8679928
##
## $`Crit-Values_prod`
##          0.90%     0.95%     0.99%
## [1,] 0.9997064 0.9999741 0.9999994
##
## $`p-value_prod`
## [1] 1
##
## $params_prod
##            ar1       ar2        ar3        ar4
## [1,] 0.4554685 0.2303401 -0.2660728 -0.2124271
```

```r
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 1.253495 mins
```

```r
start <- Sys.time()
DLMMCtest(Y, p = 4, searchType = 'GenSA')
```

```
## $`p-value_min`
## [1] 1
##
## $params_min
##        ar1        ar2        ar3        ar4
## -0.3887890  0.9762323 -0.6565886  0.4724951
##
## $`p-value_prod`
## [1] 1
##
## $params_prod
##          ar1           ar2          ar3          ar4
##  0.695537639  0.180846709 -0.178231885 -0.001950668
```

```r
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 5.046673 mins
```

Use the following command to learn more from the help tab.

```r
?DLMMCtest
```

## Optimal Test for Markov Switching Parameters

This test is developed by Carrasco Hu Ploberger (2014).

Optional Arguments are:

- p: number of AR(p) lags (default: 1 )
- N: Total number simulations (default: 3000)
- rho: bound for rho (default: 0.7)
- var_switch: indicator to consider switch in variance as well. If var_switch = 0, only mean is subjct to switch.

### Main CHP test

This version of the test requires performing a parametric bootstrap to get empirical critical values of the test and can thus take some time. To get results that are similar to those in the CHP (2014) paper set $N = 3000$. This should currently take about 2 hours when considering a switch in variance. It is much quicker when only considering switch in mean. For convinience, the example below has $var_switch = 0$ and only takes a few (~30 seconds for $N = 3000$) seconds.

```r
start <- Sys.time()
chp <- CHPtest(Y, p = 4, N = 3000, var_switch = 0)
chp
```

```
## $`Test-Stat_supTS`
## [1] 0.08435173
##
## $`Crit-Values_supTS`
##         0.90%    0.95%    0.99%
## [1,] 1.912144 2.565589 4.039641
##
## $`p-value_supTS`
## [1] 0.853
##
## $params_supTS
##           ar1       ar2        ar3         ar4
## [1,] 0.309745 0.1272577 -0.1212585 -0.08922641
##
## $`Test-Stat_expTS`
## [1] 0.6567492
##
## $`Crit-Values_expTS`
```

```
##         0.90%    0.95%    0.99%
## [1,] 1.305782 1.742489 4.161475
##
## $`p-value_expTS`
## [1] 0.75
##
## $params_expTS
##           ar1       ar2        ar3         ar4
## [1,] 0.309745 0.1272577 -0.1212585 -0.08922641
```

```
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 17.90969 secs
```

Use the following command to learn more from the help tab.

```
?CHPtest
```

### Monte-Carlo version of CHP Test

This method makes use of asymptotic distribution derived by CHP (2014) of the test under the null and is therefore much faster and takes only a few seconds regadless of the number of iterations $N$.

```
start <- Sys.time()
CHPMCtest(Y, p = 4, N = 3000)
```

```
## $`Test-Stat_supTS`
## [1] 0.08435173
##
## $`Crit-Values_supTS`
##         0.90%    0.95%    0.99%
## [1,] 1.882952 2.586179 4.171723
##
## $`p-value_supTS`
## [1] 0.84
##
## $params_supTS
##           ar1       ar2        ar3         ar4
## [1,] 0.309745 0.1272577 -0.1212585 -0.08922641
##
## $`Test-Stat_expTS`
## [1] 0.6567492
##
## $`Crit-Values_expTS`
##         0.90%    0.95%    0.99%
## [1,] 1.405798 1.919541 4.490109
##
## $`p-value_expTS`
## [1] 0.75
##
## $params_expTS
```

```
##           ar1       ar2        ar3         ar4
## [1,] 0.309745 0.1272577 -0.1212585 -0.08922641
```

```
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 1.082993 secs
```

Use the following command to learn more from the help tab.

```
?CHPMCtest
```

# Likelihood-Ratio Test

## Hensen Test

This test was developed by Hansen (1992; 1996).

Optional Arguments are:

- p: Number of AR(p) (default is 1).
- ix: List of Markov Switching parameters. 1 = just mean c(1,2) = mean and first param, (default: 1).
- iv: Idicates if Variance is also Markov Switching. 1 for true (default: 0).
- iq: Indicator that bounds q by 1-p (default: 0).
- gn: Number of different values to evaluate for each param (default: 20)
- rep: (default: 1000)

This takes about 15-20 mins.

```
start <- Sys.time()
HLRtest(Y,p = 4, iv = 1)
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
## [1] 11
## [1] 12
```

```
## $`Test-Stat`
## [1] 1.807112
##
## $`Crit-Values`
##        0.90 %   0.95 %  0.99 %
```

```
## [1,] 3.363257 3.601591 4.14071
##
## $`p-value`
## [1] 0.9456
##
## $params
##            ar1       ar2       ar3        ar4
## [1,] 0.3587903 0.1455564 -0.174976 -0.1216859
```

```
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 16.32447 mins
```

## LR Parametric Bootstrap Test

Parametric bootstrap of Likelihood-Ratio Test.

Optional Arguments are:

- ar: Number of AR(p) (default is NULL (i.e. no AR)).
- k: number of regimes (default: 2).
- N: number of simulations is N-1.

This takes only a few seconds

```
start <- Sys.time()
LR_test <-  LRBoot_ARMS(Y, ar = 4, k = 2, N = 100)
print(paste0('p-value: ',toString(round(LR_test$pval,2))))
```

```
## [1] "p-value: 0.36"
```

```
print('coefficients')
```

```
## [1] "coefficients"
```

```
LR_test$msmdl_h1$coef
```

```
##           [,1]     [,2]      [,3]       [,4]        [,5]
## [1,] 0.8415940 0.309745 0.1272577 -0.1212585 -0.08922641
## [2,] 0.3822374 0.309745 0.1272577 -0.1212585 -0.08922641
```

```
print(paste0('standard deviation: ',toString(round(LR_test$msmdl_h1$stdev,4))))
```

```
## [1] "standard deviation: 0.6668, 1.0985"
```

```
end   <- Sys.time()
print(end - start)
```

```
## Time difference of 12.38248 secs
```